

音频-视频联合文本转录

1 背景介绍

字幕，已经是如今视频中不可或缺的组成部分之一，然而制作字幕一直是一项繁重的工作，不仅耗时耗力，有时还对制作人员的语言水平提出了较高的要求。随着人工智能的迅速发展，如今自动文本转录的技术已经十分成熟了，本次作业将介绍文本转录方法是如何构建的，并且分别介绍如何基于视频和音频-视频两种模态分别完成文本转录。通过本次作业，你会学到

- 如何处理视频、音频、文本数据，将其转化为训练可用格式；
- 如何构建视频、音频的编码网络，结合不同模态的融合网络，以及针对文本的解码网络；
- 如何计算驱动网络训练的损失函数；
- 训练完成后，如何进行推理；
- 基于视频的文本转录器和基于音频-视频的文本转录器。

我们提供了完成本次作业需要的大部分代码框架，你需要按照本文档的说明完成其中部分功能，并且根据代码和实验结果回答问题，撰写报告。

2 作业说明

提交方式：每个小组需要提交一个单独的 ZIP 文件，命名为“大作业 _ 小组编号.zip”。压缩包中需包含报告、可运行的代码、训练好的模型以及测试的脚本。

模型文件的提交方式：1. 随压缩包一起提交（不推荐）。2. 保留在我们提供的服务器中，并提供绝对路径（请做好备份）。3. 上传清华云盘并提供链接。

报告内容：完成本文档中要求的内容，并写出每位小组成员在大作业中的具体工作。

评分标准：

- 基本实现 40%：实现基本框架，代码正常运行并取得合理结果
- 进阶实现 20%：完成进阶训练方法实现，相关代码正常运行
- 分析创新 20%：进行实验对比分析，或提出改进方案并进行验证
- 实验报告 20%：按要求完成报告，包括实验汇报和文字简述题

3 环境配置

3.1 （推荐）个人配制

由于公共服务器资源有限，我们建议有计算资源的小组使用自己的计算资源，避免特定时间集中使用导致冲突。本次作业单个实验显存占用量小于 5G，使用 4090 显卡运行时间不超过 6 小时。

本次作业所需的环境及数据已经上传至清华云盘，[链接点击此处](#)，包含以下文件：

- `final_project_conda_env.tar.gz`: 使用 `conda pack` 打包的 `conda` 环境，可参考[此文档](#)解压安装。你也可以参考我们提供的 `requirements.txt` 文件手动安装。
- `final_project_data.zip`: 训练和测试数据集。
- `final_project_ckpt.zip`: 预训练模型参数。

3.2 (备选) 公共服务器

请认真阅读服务器使用说明并严格按照说明使用。激活 `conda` 环境可使用：

```
conda activate final_project
```

数据和预训练模型分别位于 `/data2/final_project_data` 和 `/data2/final_project_ckpt`。

4 基本框架实现

4.1 基于视频的文本转录

我们首先从更加简单的基于视频的文本转录任务开始。基于视频的文本转录也被称为唇读(lip reading)，即只根据人物在说话时的嘴唇变化视频来解读文本内容，这个任务非常有趣，但是即使是对人来说也是非常具有挑战性的。

4.1.1 数据处理

该任务需要人物说话的嘴唇变化视频作为输入，对应文本作为标签。我们已经对每个视频做过预处理，将其中人物的嘴唇附近区域截取出来保存为对应的视频。现在，给定视频路径和对应的文本路径，你需要加载视频和文本，并将其处理成神经网络可以使用的格式，具体任务如下。

视频本身的数据格式较为规整，因此它的加载处理方式比较简单，通常我们只需要读取视频，将其储存为多维 `tensor`，最后再进行一些归一化即可。请按照以下说明完成 `dataset.py` 文件中的 `AudioVideoDataset` 类的 `load_video` 方法：

1. 根据给定的 `video_name`，读取视频（你可以使用 `cv2.VideoCapture` 方法来完成这一过程）；
2. 将每一帧变为单通道的灰色图像（你可以使用 `cv2.cvtColor` 来完成，请注意 `opencv` 在加载图像时，默认的通道顺序为 `BGR`）；
3. 将视频转化为 `numpy array`，并且对其进行视频数据变换（数据变换被定义在 `self.transform` 中）；
4. 输出的特征维度为 `(T, H, W, 1)`，`T` 表示视频帧数，`H` 和 `W` 分别表示图像的高宽，`1` 是人为增加的维度，方便后续处理。

Task 1

实现 `AudioVideoDataset` 类的 `load_video` 方法。

文本数据通常需要更加细致的处理。因为文本不像图像有天然的数值表示 (`RGB`)，在对文本进行计算之前，我们需要为其设计一种便于使用的数值形式。现在通用的做法分为两步：首先切分文本，将

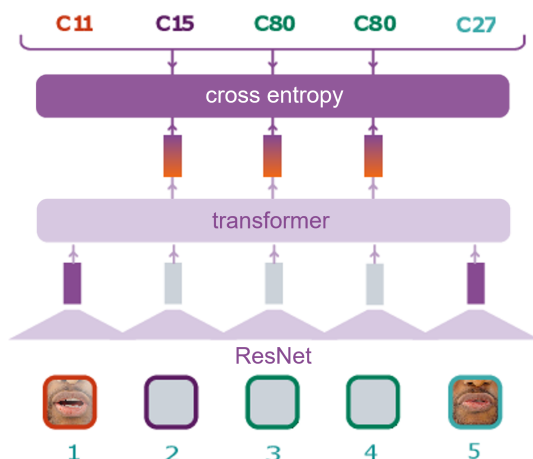


图 1: 基于视频的文本转录模型

其划分成离散的词符 (token); 然后对不同的词符建立词表 (vocabulary), 以整数来代表词符。其中第一步分词 (tokenization) 最为关键, 简单的基于字母的分词或者基于单词的分词都存在问题, 大家使用最多的是一种叫做 byte-pair-encoding (BPE) 的分词技术。BPE 的代码实现已经非常成熟, 这里我们使用 sentencepiece 库提供的实现版本。

请按照以下说明完成 dataset.py 文件中的 AudioVideoDataset 类的 get_label 方法:

1. 根据给定的 index, 读取标签文本。你可以通过 index 在 self.label_offsets_list 查询需要的文本在标签文本中的位置, 得到元组 (s, e) 表示起始和终止位置。你可以使用 python 文本 IO 的 seek 和 read 函数来读取对应的部分;
2. 对标签文本进行分词 (你可以使用 self.bpe_tokenizer 来完成, 参考[这部分代码](#));
3. 对分词后的文本进行编码 (你可以使用 self.dictionary 来完成, 参考[这部分代码](#), 注意如果某个词符不在词表中, 请不要向词表中加入新词);
4. 输出的标签是维度为 (L) 的长整型数组, L 表示词符个数。

Task 2

实现 AudioVideoDataset 类的 get_label 方法。

阅读 BPE 的论文 ([论文链接](#)), 简要说明其构建过程, 以及其相比基于字母的分词和基于单词的分词的优势。

4.1.2 模型构建

接下来可以开始构建模型。我们采用的模型结构如图 1 所示, 首先利用 resnet 网络对每一帧图像进行编码, 抽取每帧图像的全局特征; 接着用 transformer 模型将图像特征映射到文本空间, 因为文本标签已经转化为整数, 最后直接利用交叉熵作为损失函数驱动训练。

ResNet 网络是通用的视觉信号处理模型, 通过著名的残差连接能够有效训练深层网络, 抽取视觉特征。请按照以下说明完成 modality_encoder.py 文件中的 BasicBlock 类和 ResNet 类:

1. BasicBlock 类是堆叠 ResNet 网络的最小单元, 他的基本结构如图 2 所示, 注意残差连接通常如左图所示是一个直连, 但如果特征的维度发生改变, 则会变为右图;

2. 请完成 BasicBlock 类的 `__init__` 函数, inplanes, planes, stride 为 conv1 的输入通道, 输出通道, 以及步长; conv2 使用 planes 作为输入输出通道数, 1 作为步长即可; 为了性能更优以及与预训练模型保持一致, 图中的 ReLU 激活函数实际使用带可学习参数的 PReLU 激活函数, 参数数量设置为 planes;
3. 请完成 BasicBlock 类的 forward 函数, 注意需要处理图中展示的两种情况;
4. 请完成 ResNet 类的 `_make_layer` 函数, 该函数负责构建 resnet 网络中的一个 stage, block 为 BasicBlock 类, planes 为该 stage 的输出通道数, blocks 为该 stage 的 block 数量, stride 为该 stage 的降采样率 (降采样通过 conv 实现, 所以也是步长)。请注意每个 stage 的输入通道数需要自己计算, 最开始的输入通道数为 self.inplanes。

Task 3

实现 BasicBlock 类的 `__init__` 方法和 forward 方法, 与 ResNet 类的 `_make_layer` 方法。

提示: 我们提供了预训练的特征提取器 (详见第 4.1.4 章)。如果你的实现是正确的, 预训练的特征提取器可以正确加载, 即每个参数的名称和尺寸是对应的。你可以关注加载预训练 checkpoint 时返回的信息, 也可以将预训练 checkpoint 中的参数名称和尺寸打印出来, 来确认你的实现是否正确。

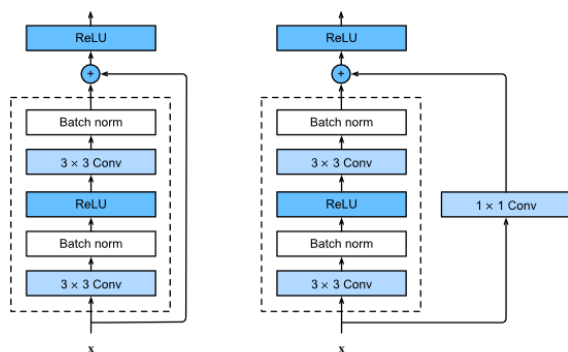


图 2: ResNet 网络的 BasicBlock

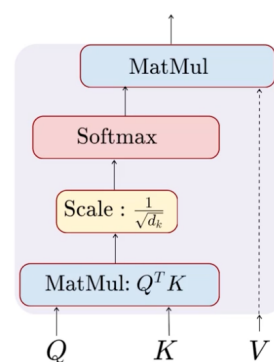


图 3: Transformer 网络的 attention 计算

Transformer 网络从自然语言处理领域诞生, 因其通用性逐渐地被拓展到其他场景中。Transformer 网络通常由 encoder-decoder 两部分组成, encoder 对输入信号做进一步的编码处理, decoder 则根据 encoder 的输出来解码得到预测。两部分网络虽然功能上有所区分, 但都是建立在 multi-head attention 这一基础模块上。

multi-head attention 的通用计算表示为图 3, Q 表示 query, K 和 V 表示 key 和 value。请按照以下说明完成 `multihead_attention.py` 文件中的 `MultiheadAttention` 类:

1. 按照图示完成 attention 的计算过程, 代码中 `attn` 表示最后的输出, `attn_weights` 表示 attention 的权重矩阵;
2. 在原始的计算过程中, 加入对 `attn_mask` 的处理, `attn_mask` 表示 query (对应行) 能够看到哪些 key (对应列), 如果输入有 `attn_mask`, 它的矩阵元素要么是 0 要么是 -inf, 0 的位置表示对应的 query 可以看到 key, 而 -inf 的位置表示对应的 query 不能看到可以。请结合 `attn_mask` 使得 query 不会使用指定 key 的信息;

3. 在原始的计算过程中, 加入对 `key_padding_mask` 的处理, `key_padding_mask` 指明了哪些输入只是起到填充占位的作用, 在 `attn` 的计算时不应该考虑这些符号。如果输入中有 `key_padding_mask`, 它的矩阵元素由 0、1 组成, 0 的位置表示非占位符, 1 的位置表示占位符。请结合 `key_padding_mask`, 保证 `attn` 的计算过程不会使用占位符的特征;
4. 在上述任务中, 请注意对 `multihead` 的处理, 最开始使用的 Q, K, V 的维度为 $(\text{batch_size} \times \text{num_head}, \text{seq_len}, \text{feat_dim})$;
5. 如果需要返回注意力权重 `attn_weights`, 即 `need_weights` 为 `True` 时, 你需要返回所有 head 的注意力权重的平均值, 维度为 $(\text{batch_size}, \text{tgt_len}, \text{src_len})$ 。

Task 4

实现 `MultiheadAttention` 类的 `forward` 方法。

4.1.3 损失函数

本次项目使用交叉熵损失函数来训练网络。请按照以下说明完成 `losses/label_smoothed_cross_entropy.py` 文件中的 `label_smoothed_nll_loss` 函数:

1. 阅读 `pytorch` 关于交叉熵损失函数的介绍[链接](#), 实现基本的交叉熵计算, 代码中 `lprobs` 表示 `log probability`, `target` 表示标签, `reduce` 表示是否要对所有 `loss` 求和;
2. 在原始的计算过程中, 加入对 `ignore_index` 的处理, `ignore_index` 表示需要忽略的 `target` 的取值, 不加入到 `loss` 计算中;
3. 在原始的计算过程中, 加入标签平滑 (`label smooth`), 标签平滑是一种有效防止过拟合, 提升模型泛化能力的技术, 请阅读论文[链接](#)中和标签平滑相关的部分, 并且实现代码;
4. 该函数返回一个元组: 平滑后的 `loss`, 标准的 `nll loss`。如果参数传入 `reduce=True`, 需要将所有 `token` 位置的 `loss` 相加得到一个 `scalar`。
5. 在上述任务中, 请不要使用 `pytorch` 提供的交叉熵损失函数。

Task 5

实现 `label_smoothed_nll_loss` 函数。

4.1.4 模型训练

训练的启动脚本为 `scripts/train_v2t.sh`, 需要指定如下参数并注意相关说明:

- `DATA_DIR`: 数据集路径, 对应数据集目录下的 `30h_data` 文件夹。如果使用公共服务器, 直接设置为 `/data2/final_project_data/30h_data`;
- `TOKENIZER_PATH`: 分词器路径, 对应数据集目录下的 `spm1000/spm_unigram1000.model` 文件。如果使用公共服务器, 设置为 `/data2/final_project_data/spm1000/spm_unigram1000.model`;
- `PRETRAIN_PATH`: 预训练模型路径, 对应预训练模型目录下的 `pretrained_model.pth` 文件。如果使用公共服务器, 直接设置为 `/data2/final_project_ckpt/pretrained_model.pth`;
- `hydra.run.dir` 表示实验结果的储存路径, 可以自由设置, 请注意训练程序会自动读取实验目录下的 `checkpoint` (如果存在) 并继续训练, 如果你想从头开始训练, 请改变实验目录或者删除原有的 `checkpoint`。

Task 6

完成基于视频的文本转录任务训练，绘制训练和测试的损失函数与准确率曲线。

4.2 基于音频-视频的文本转录

在视频的基础上，我们还可以利用音频信息进行文本转录。基于音频-视频信息的文本转录可以取得很好的效果，基本能够满足自动字幕生成的要求。这一任务只需要在基于视频的文本转录模型基础上加入音频信息即可完成。

4.2.1 数据处理

音频数据的处理比较特殊，大多数方法不会将原始的音频信号直接作为网络输入，而是将其频谱作为输入，其中变换方式也非常多样（比如梅尔谱）。请按照以下说明完成 `dataset.py` 文件中的 `AudioVideoDataset` 类的 `load_audio` 方法：

1. 根据 `audio_name` 加载音频数据（你可以使用 `wavfile` 来完成）；
2. 使用 `logfbank` 算法提取音频特征（你可以使用 `python_speech_features_cuda` 来完成。在使用 `logfbank` 时将数据类型转为 `cupy.ndarray`，使用 GPU 计算来减轻 CPU 的负担）；
3. 将每 4 帧（由 `stack_order_audio` 参数确定）的音频特征拼起来组成这 4 帧对应时间的共同特征，如果总帧数不是 4 的倍数，少的部分补零；
4. 最后返回的音频特征维度应为 (T, C) ， T 表示帧数（每 4 帧拼起来后的数量）， C 表示特征维度。

Task 7

实现 `AudioVideoDataset` 类的 `load_audio` 方法。

调研 `logfbank` 算法，简要说明其计算过程。

4.2.2 模型构建

处理音频的网络结构没有视频的网络那么复杂，通常只需要一层线性层来调整特征的维度。请按照以下说明完成 `modality_encoder.py` 文件中的 `AudioEncoder` 类：

1. 完成 `__init__` 方法，创建线性层，输入维度为 `audio_feat_dim`，输出维度为 `encoder_embed_dim`；
2. 完成 `forward` 方法，请注意输入的维度为 (B, C, T) ， B 表示批次大小， C 表示特征维度， T 表示帧数。

Task 8

实现 `AudioEncoder` 类。

得益于 Transformer 的通用性，模态融合模块不需要改动，只需要把输入的音频特征和视频特征拼起来即可，之后的网络结构和原来是完全一样的。

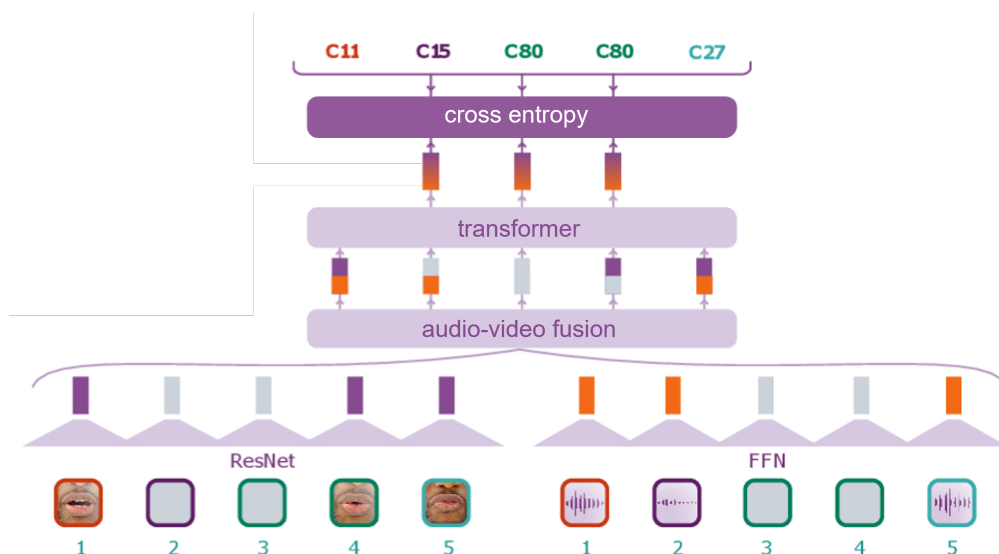


图 4: 基于音频-视频的文本转录模型

4.2.3 模型训练

模型训练脚本为 `scripts/train_av2t.sh`，参数指定与章节 4.1.4 类似。

Task 9

完成基于音频和视频的文本转录任务训练，绘制训练和测试的损失函数与准确率曲线。

4.3 模型推理

模型推理通过自回归的方式完成，即每次生成一个词符，生成之后将该词符加入已经生成的内容中作为 decoder 部分的输入继续生成下一词符。这种生成方式虽然速度快，但是可能陷入局部最优解，因此通常使用的解码方式是一种叫做 beam search 的搜索算法。

Task 10

阅读 `sequence_generator.py` 的 `_generate` 方法，简要说明 beam search 的具体过程。

推理的启动脚本为 `scripts/test_v2t.sh`（视频的文本转录）和 `scripts/test_av2t.sh`（音频-视频文本转录），需要指定如下参数：

- CHECKPOINT_PATH: 训练好的模型 checkpoint 的路径，最好使用绝对路径。
- RESULT_PATH: 保存推理结果的路径，最好使用绝对路径。

Task 11

完成基于视频、基于音频和视频的文本转录推理，分析推理结果，以及推理准确率和之前得到的测试准确率的差异。

4.4 参考结果

我们在表格 1 中提供了 Baseline 结果供参考。当正确完成以上任务后，你应该能得到接近的结果。

模型	train loss	train acc	valid loss	valid acc	inference WER
视频	59.95	67.85%	63.71	66.65%	46.46
视频 + 音频	33.91	94.40%	37.33	91.55%	15.17

表 1: 基本框架的 Baseline 参考

5 进阶方法实现

你需要从以下三种备选方法中，**选择至少两种**进行实现。三种方法分别从训练目标、音频-视频模态鲁棒性、特征层正则化三个角度改进模型。实现完成后，请使用对应训练脚本完成训练，并使用对应测试脚本完成推理，并报告结果。

5.1 损失函数改进

基础实现中已经完成了带标签平滑的交叉熵损失。在这一进阶任务中，你需要在其基础上加入可选的 focal loss 调制项和 confidence penalty 正则项，用来改变模型对不同难度样本以及过度自信预测的处理方式。

请按照以下说明完成 losses/label_smoothed_cross_entropy.py 文件中的 LabelSmoothedCrossEntropyCriterion 类的 compute_enhanced_loss 方法：

1. 当 self.focal_gamma 大于 0 时，使用 $p_t = \exp(-\text{nll_loss})$ 估计目标词符概率，并将每个 token 的 loss 乘以 $(1 - p_t)^\gamma$ ；
2. 当 self.confidence_penalty 大于 0 时，根据 lprobs 计算每个位置的 $\sum_i p_i \log p_i$ ，其中 $p_i = \exp(\text{lprobs}_i)$ ，再乘以 self.confidence_penalty 后加入 loss；
3. 如果 reduce=True，需要将 loss 和 nll_loss 都对所有 token 求和后返回；如果 reduce=False，则保留逐 token 结果。

Task 12

完成 compute_enhanced_loss 方法的实现及相关实验。

使用 scripts/train_enhanced_loss_v2t.sh 和 scripts/test_enhanced_loss_v2t.sh 完成基于视频的文本转录训练与推理。脚本采用以下默认参数配置，在此阶段可不用修改：

```
criterion.focal_gamma=1.0
criterion.confidence_penalty=0.0
```


5.2 Modality Dropout

音频-视频文本转录模型在训练时同时接收音频和视频。如果模型过度依赖其中一种模态，在测试时遇到噪声或模态质量下降时可能表现不稳定。Modality dropout 在训练阶段随机遮蔽部分样本的某一种模态，鼓励模型同时利用两种模态并提升鲁棒性。

请按照以下说明完成 `models/encoder_backbone.py` 文件中的 `apply_modality_dropout` 方法：

1. 对 batch 中的每个样本独立采样，按照 `self.modality_dropout` 决定该样本是否执行 modality dropout；
2. 对于被选中的样本，按照 `self.audio_dropout` 决定遮蔽音频还是遮蔽视频。未被选中的样本必须保持不变；
3. 当 `self.modality_dropout_mode` 为 `sample` 时，遮蔽被选中样本的一整个模态，即该样本对应模态的所有通道和所有时间步都置为 0；
4. 当 `self.modality_dropout_mode` 为 `span` 时，只遮蔽被选中模态中的一个连续时间片段。片段长度需要在 `self.modality_dropout_span_min` 和 `self.modality_dropout_span_max` 之间随机采样，并根据当前序列长度 `T` 裁剪到合法范围。

Task 13

完成 `apply_modality_dropout` 方法的实现及相关实验。

使用 `scripts/train_modality_dropout_av2t.sh` 和 `scripts/test_modality_dropout_av2t.sh` 完成音频-视频文本转录训练与推理。脚本采用以下默认参数配置，在此阶段可不用修改：

```
model.modality_dropout=0.05
model.audio_dropout=0.3
model.modality_dropout_mode=span
model.modality_dropout_span_min=10
model.modality_dropout_span_max=30
```

5.3 Feature Mask

Feature mask 在融合后的特征序列上进行遮蔽，是一种类似 SpecAugment 和 HuBERT 预训练 mask 策略的正则化方法。它不改变输入数据本身，而是在进入 Transformer encoder 前对时间维度或通道维度的特征片段置零，从而降低模型对局部特征的依赖。

请按照以下说明完成 `models/encoder_backbone.py` 文件中的 `apply_feature_mask` 方法：

1. 当 `self.mask_prob` 大于 0 时，沿时间维度采样 mask 片段，可以调用 `utils/model_utils.py` 中的 `compute_mask_indices` 函数。得到 mask 后，将对应时间步上的所有通道置为 0；
2. 当 `self.mask_channel_prob` 大于 0 时，沿通道维度采样 mask 片段，可以调用 `compute_mask_indices` 函数。通道 mask 会遮蔽被选中的通道，并作用于所有时间步；
3. 时间 mask 和通道 mask 彼此独立，可以只启用其中一种，也可以同时启用。

Task 14

完成 `apply_feature_mask` 方法的实现及相关实验。

使用 `scripts/train_feature_mask_v2t.sh` 和 `scripts/test_feature_mask_v2t.sh` 完成基于视频的文本转录训练与推理。脚本采用以下默认参数配置，在此阶段可不用修改：

```
model.apply_mask=true
model.mask_prob=0.05
model.mask_length=10
model.mask_channel_prob=0.0
```

6 实验分析和优化

本章节将进行深入的实验分析和方法的创新优化。你需要在以下两个任务路线中**选择一项**完成。

6.1 路线一：进阶方法对比分析

你需要基于第五章实现的进阶方法，设计额外实验来分析不同方法或不同配置的作用，至少满足以下要求：

1. 至少选择第五章中的一种进阶方法进行深入分析，可以比较不同方法之间的差异，也可以固定一种方法后比较不同超参数配置；
2. 报告中需要清晰说明实验设置、对照组、关键实验结果和结论；
3. 除了结果数值的高低变化，还可以结合方法机制、训练曲线、样本可视化等对实验进行解释分析。

Task 15

选择第五章中的一种或多种进阶方法，完成不同方法或不同配置的实验对比。

6.2 路线二：提出新的优化方法

你需要提出一种不同于第五章三种方法的新优化方案，并通过实验验证其有效性。新的方法可以是模型结构、训练策略、数据增强、损失函数、解码策略或多模态融合方式上的改进。报告中需要包含以下内容：

1. 方法动机：说明你观察到的瓶颈或假设；
2. 方法设计：清楚描述新增或修改的模块、公式、训练策略或推理策略；
3. 实验验证：报告模型训练和测试情况，并与对应基线进行比较；
4. 结果分析：说明该方法对性能、计算开销等方面的作用，以及失败或收益有限时的可能原因。

Task 16

提出一种新的模型优化方法并完成实验验证。